

Rút trích dữ liệu và tiền xử lý dữ liệu Pima Indians Diabetes

Báo cáo học phần thay thế khóa luận

Nhóm 01

Ngày 3 tháng 5 năm 2026

Khoa Toán Ứng dụng - Trường Đại học Sài Gòn

Mục tiêu buổi trình bày

- Trình bày đầy đủ quy trình nghiên cứu từ dữ liệu đến kết quả mô hình.
- Giải thích rõ các quyết định kỹ thuật (missing ản, imputation, scaling, baseline).
- Tổng hợp insight thực nghiệm để trả lời trực tiếp các câu hỏi nghiên cứu.
- Đưa ra kết luận, hạn chế và khuyến nghị triển khai trong thực tế.

Chương 1 - Tổng quan vấn đề

Chương 2 - Cơ sở lý thuyết và nghiên cứu liên quan

Chương 3 - Dữ liệu và phương pháp đề xuất

Chương 4 - Thực nghiệm và kết quả đạt được

Chương 5 - Kết luận và khuyến nghị

Chương 1 - Tổng quan vấn đề

- Đái tháo đường là bệnh mạn tính có xu hướng gia tăng và nhiều biến chứng nặng.
- Nhu cầu sàng lọc sớm đòi hỏi cách tiếp cận định lượng, tái lập và nhất quán.
- Dữ liệu y sinh thực tế có nhiều, missing ẩn và quan hệ đa biến phức tạp.
- Học máy phù hợp để mô hình hóa xác suất nguy cơ, hỗ trợ quyết định ban đầu.

Bộ dữ liệu lựa chọn: Pima Indians Diabetes (benchmark kinh điển).

Mục tiêu và câu hỏi nghiên cứu

Mục tiêu chính

- Xây dựng pipeline phân tích dữ liệu y sinh có khả năng tái lập.
- So sánh các chiến lược *imputer + scaler + model* trong cùng điều kiện.
- Tối ưu quyết định vận hành thông qua phân tích threshold trade-off.

3 câu hỏi cốt lõi

1. *zero-as-missing* có cải thiện kết quả không?
2. Tổ hợp preprocessing + model nào phù hợp nhất?
3. Precision/Recall biến thiên thế nào khi thay đổi threshold?

Đối tượng, phạm vi và luồng tổng quát

- Dữ liệu: 768 mẫu, 8 biến đầu vào, nhãn nhị phân outcome.
- Phạm vi: EDA, tiền xử lý, baseline (Logistic Regression, Random Forest).
- Không bao gồm: deep learning, external dataset, tuning quy mô lớn.

Luồng kỹ thuật

Khảo sát dữ liệu → xử lý missing ẩn → EDA → pipeline tiền xử lý → huấn luyện baseline → đánh giá đa chỉ số và phân tích ngưỡng.

Chương 2 - Cơ sở lý thuyết và nghiên cứu liên quan

- Bài toán phân lớp nhị phân: dự đoán $\text{outcome} \in \{0, 1\}$.
- Dữ liệu tabular y sinh có:
 - phân phối lệch,
 - outlier và missing ẩn,
 - tương tác biến không thuần tuyến tính.
- Hiệu năng mô hình phụ thuộc mạnh vào chất lượng tiền xử lý.

Xử lý dữ liệu: zero-as-missing, imputation, scaling

Zero-as-missing

- Chuyển các giá trị 0 phi sinh lý ở glucose, blood_pressure, skin_thickness, insulin, bmi thành missing.

Imputation

- Median, KNN, Iterative (MICE).

Scaling

- StandardScaler và MinMaxScaler để đồng nhất thang đo.

Mô hình baseline và chỉ số đánh giá

Mô hình

- Logistic Regression: baseline tuyến tính, dễ diễn giải.
- Random Forest: mô hình phi tuyến, bắt tương tác biến tốt.

Chỉ số

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Lưu ý

Với bài toán sàng lọc, Recall và F1 thường quan trọng hơn Accuracy đơn lẻ.

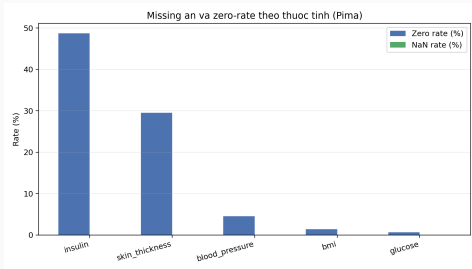
Chương 3 - Dữ liệu và phương pháp đề xuất

Nguồn dữ liệu và thuộc tính

- `Lab_03/data/pima-indians-diabetes.csv`
- `Lab_03/data/pima-indians-diabetes.names`

Biến tiêu biểu	Ý nghĩa
glucose	Nồng độ glucose huyết tương
insulin	Nồng độ insulin huyết thanh
bmi	Chỉ số khối cơ thể
age	Tuổi bệnh nhân
outcome	Nhãn phân lớp (0/1)

Kiểm tra chất lượng dữ liệu



Nhận xét chính

- Không có dòng trùng lặp.
- Không có NaN tường minh trong file gốc.
- Missing ẩn tập trung ở `insulin`, `skin_thickness`.
- Cần chuyển 0 bất thường thành missing trước huấn luyện.

Code block: chia dữ liệu có stratify

```
1 X = df[feature_columns].copy()
2 y = df["outcome"].astype(int)
3 X[zero_as_missing_columns] = X[zero_as_missing_columns].replace(0,
4     np.nan)
5 X_train_val, X_test, y_train_val, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42, stratify=y
7 )
8 X_train, X_val, y_train, y_val = train_test_split(
9     X_train_val, y_train_val, test_size=0.25,
10    random_state=42, stratify=y_train_val
11 )
```

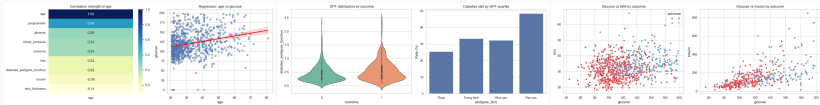
Insight: Chia 2 bước giúp đạt đúng tỉ lệ 60/20/20 và giữ ổn định tỷ lệ lớp dương trên mọi tập.

Code block: pipeline baseline

```
1 Pipeline(steps=[  
2     ("imputer", SimpleImputer(strategy="median")),  
3     ("scaler", StandardScaler()),  
4     ("clf", LogisticRegression(max_iter=5000, random_state=42))  
5 ])
```

- Tất cả phép biến đổi được fit trên train.
- Giảm data leakage và tăng tính tái lập.
- Dễ mở rộng sang nhiều cấu hình khác nhau.

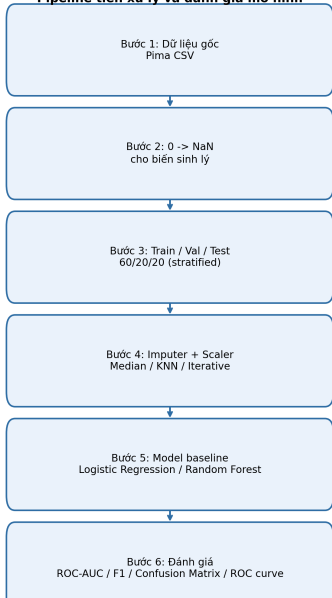
EDA chuyên sâu theo nhóm biến



- glucose thể hiện khả năng phân tách lớp rõ nhất.
- DPF, BMI, insulin bổ sung tín hiệu rủi ro khi kết hợp đa biến.
- Quan hệ không thuần tuyến tính $\Rightarrow$ cần so sánh mô hình phi tuyến.

Pipeline tiền xử lý đề xuất

Pipeline tiền xử lý và đánh giá mô hình



1. Chuẩn hóa dữ liệu đầu vào.
2. Xử lý missing ẩn.
3. Imputation.
4. Scaling.
5. Huấn luyện + đánh giá.

Thông điệp: Quy trình thống nhất giúp so sánh công bằng và giảm lỗi thao tác thủ công.

Chương 4 - Thực nghiệm và kết quả đạt được

Chiến lược chia dữ liệu

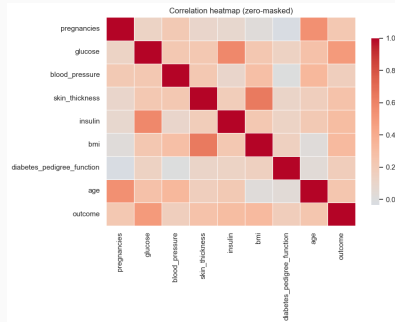
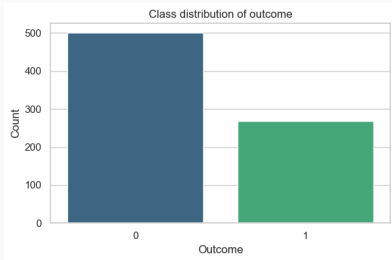
- Train: 460 mẫu
- Validation: 154 mẫu
- Test: 154 mẫu

Không gian so sánh

- Imputation: Median, KNN, Iterative
- Scaling: Standard, MinMax
- Model: Logistic Regression, Random Forest

Tổng cộng: 12 cấu hình trên cùng seed và cùng split.

Phân bố lớp và tương quan chính



- Lớp âm chiếm ưu thế nhẹ $\Rightarrow$ cần theo dõi Recall/F1.
- glucose có tương quan mạnh nhất với outcome.
- Không có biến đơn lẻ đủ quyết định nhãn $\Rightarrow$ cần mô hình đa biến.

Code block: đánh giá một cấu hình

```
1 def evaluate_strategy(imputer_name, scaler_name, model_name):
2     pipe = Pipeline([...])
3     pipe.fit(X_train, y_train)
4     y_val_pred = pipe.predict(X_val)
5     y_val_prob = pipe.predict_proba(X_val)[: , 1]
6     y_test_pred = pipe.predict(X_test)
7     y_test_prob = pipe.predict_proba(X_test)[: , 1]
8     return {
9         "val_f1": f1_score(y_val, y_val_pred),
10        "val_roc_auc": roc_auc_score(y_val, y_val_prob),
11        "test_f1": f1_score(y_test, y_test_pred),
12        "test_roc_auc": roc_auc_score(y_test, y_test_prob),
13    }
```

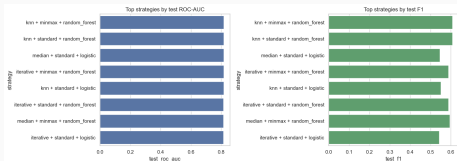
Insight: Mọi tổ hợp dùng cùng pipeline đánh giá, đảm bảo tính công bằng trong bảng xếp hạng.

Kết quả 12 cấu hình (tóm tắt)

Imputer	Scaler	Model	Test AUC	F1
KNN	MinMax	Random Forest	0.8170	0.6078
KNN	Standard	Random Forest	0.8168	0.6078
Median	Standard	Logistic Reg.	0.8167	0.5455
Iterative	MinMax	Random Forest	0.8156	0.5882

- Nhóm Random Forest ổn định hơn về F1.
- KNN Imputer thường xuất hiện ở top theo ROC-AUC.

So sánh chiến lược và cấu hình đề xuất



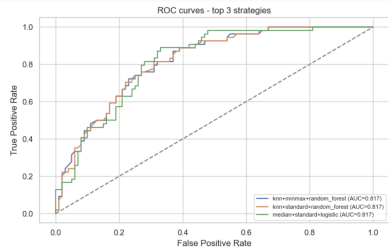
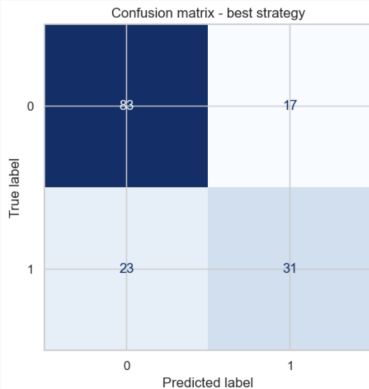
Cấu hình đề xuất

- KNN + MinMax + Random Forest
- Val AUC: 0.8700
- Test AUC: 0.8170
- Test F1: 0.6078
- Test Recall: 0.5741

Nhận xét

- AUC top khá sát nhau.
- Cần chọn theo mục tiêu nghiệp vụ, không chỉ theo 1 chỉ số.

Phân tích sai số: Confusion matrix + ROC



- Tại ngưỡng 0.5: TN=86, FP=17, FN=23, TP=31.
- Vẫn bỏ sót ca dương đáng kể $\Rightarrow$ cần tối ưu threshold cho sàng lọc.

Code block: quét threshold

```
1 thresholds = np.arange(0.30, 0.75, 0.05)
2 rows = []
3 for thr in thresholds:
4     pred_thr = (best_artifacts["y_test_prob"] >= thr).astype(int)
5     rows.append({
6         "threshold": thr,
7         "precision": precision_score(y_test, pred_thr),
8         "recall": recall_score(y_test, pred_thr),
9         "f1": f1_score(y_test, pred_thr),
10    })
11 threshold_df = pd.DataFrame(rows)
```

Trade-off theo threshold

Threshold	Precision	Recall	F1
0.30	0.5789	0.8148	0.6769
0.40	0.6167	0.6852	0.6491
0.50	0.6458	0.5741	0.6078
0.60	0.6944	0.4630	0.5556
0.70	0.7500	0.3333	0.4615

Kết luận vận hành

Ngưỡng 0.30 phù hợp hơn cho pha sàng lọc ban đầu vì Recall cao (0.8148), chấp nhận đánh đổi Precision.

Chương 5 - Kết luận và khuyến nghị

1. *zero-as-missing* có tác động thực chất đến chất lượng mô hình.
2. KNN + MinMax + Random Forest cho test ROC-AUC cao nhất.
3. Random Forest ổn định hơn Logistic Regression về F1 trong dữ liệu phi tuyến.
4. Điều chỉnh threshold giúp cải thiện mạnh mẽ tiêu chuẩn sàng lọc.

Q1 – Zero-as-missing có hữu ích không?

Có, giúp giảm nhiễu từ giá trị phi sinh lý và ổn định AUC quanh 0.81 trên test.

Q2 – Tổ hợp tốt nhất?

KNN Imputer + MinMaxScaler + Random Forest.

Q3 – Trade-off theo threshold?

Giảm threshold từ 0.5 xuống 0.3 làm Recall tăng rõ ($0.5741 \rightarrow 0.8148$), F1 cũng tăng ($0.6078 \rightarrow 0.6769$).

Đóng góp, hạn chế và khuyến nghị

Đóng góp

- Chuẩn hóa pipeline tái lập từ EDA đến đánh giá baseline.
- Cung cấp benchmark 12 cấu hình trong cùng điều kiện thực nghiệm.

Hạn chế

- Dữ liệu nhỏ (768 mẫu), chưa có external validation.
- Chưa tuning hệ thống và chưa calibration xác suất.

Khuyến nghị

- Ngắn hạn: Stratified K-Fold + tuning + calibration.
- Trung hạn: thử boosting và kỹ thuật cân bằng lớp.
- Dài hạn: mở rộng dữ liệu, đóng gói pipeline triển khai.

Phụ lục

Bảng đầy đủ 12 cấu hình (appendix)

Imputer	Scaler	Model	Test AUC	F1
KNN	MinMax	Random Forest	0.8170	0.6078
KNN	Standard	Random Forest	0.8168	0.6078
Median	Standard	Logistic Regression	0.8167	0.5455
Iterative	MinMax	Random Forest	0.8156	0.5882
KNN	Standard	Logistic Regression	0.8154	0.5510
Iterative	Standard	Random Forest	0.8153	0.5882
Median	MinMax	Random Forest	0.8149	0.5941
Iterative	Standard	Logistic Regression	0.8148	0.5417
Median	Standard	Random Forest	0.8136	0.5941
KNN	MinMax	Logistic Regression	0.8117	0.5474
Median	MinMax	Logistic Regression	0.8098	0.5532
Iterative	MinMax	Logistic Regression	0.8093	0.5567

Cảm ơn thầy/cô và các bạn đã theo dõi.